

DATA VISUALISATION WITH PYTHON

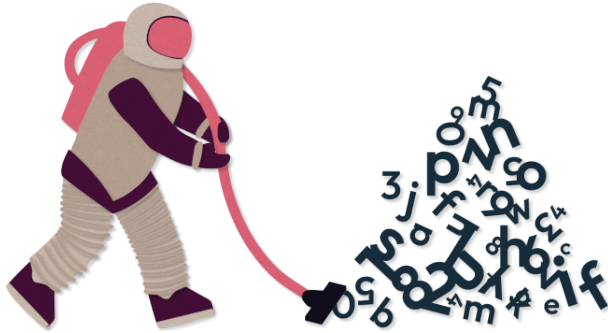


**Jumping
Rivers**

jumpingrivers.com

CREATING CLARITY WITH YOUR DATA.

Saving organisations like yours thousands of work hours.



Jumping Rivers is an analytics company whose passion is data and machine learning. We help our clients move from data storage to data insights. Our trainers and consultants come with over 100 years combined experience in R, Python, Stan, Scala and other programming languages.

Consultancy

- Posit (formerly RStudio) certified - the only UK partner.
- We have plans for managing all Posit products. From on-demand support to full care packages.
- As a Posit reseller, we can provide free consultancy.
- Experts in both data science and data engineering.
- Creation of bespoke dashboards and Shiny apps.
- Optimisation of your algorithms.



Training

- R/Python/Stan/Scala
- The Tidyverse
- Machine Learning and Tensorflow
- Dashboards with Shiny
- Statistical Modelling
- And many more!



hello@jumpingrivers.com

@jumping_uk

“OFTEN THE MOST EFFECTIVE WAY TO DESCRIBE, EXPLORE, AND SUMMARIZE A SET OF NUMBERS—EVEN A VERY LARGE SET—IS TO LOOK AT PICTURES OF THOSE NUMBERS. FURTHERMORE, OF ALL METHODS FOR ANALYZING AND COMMUNICATING STATISTICAL INFORMATION, WELL-DESIGNED DATA GRAPHICS ARE USUALLY THE SIMPLEST AND AT THE SAME TIME THE MOST POWERFUL.”

— EDWARD TUFTE, *THE VISUAL DISPLAY OF QUANTITATIVE INFORMATION*.

Introduction

Over the last few decades, Python has risen to prominence as one of the most widely-used programming languages by statisticians and data scientists. In that time, a plethora of popular graphics packages have been developed to tackle a range of visualisation problems, from static 2D plots to 3D animations.

So much choice can feel overwhelming at a first glance (see Figure 1)! However, the libraries can largely be grouped into a few categories which we'll outline here.

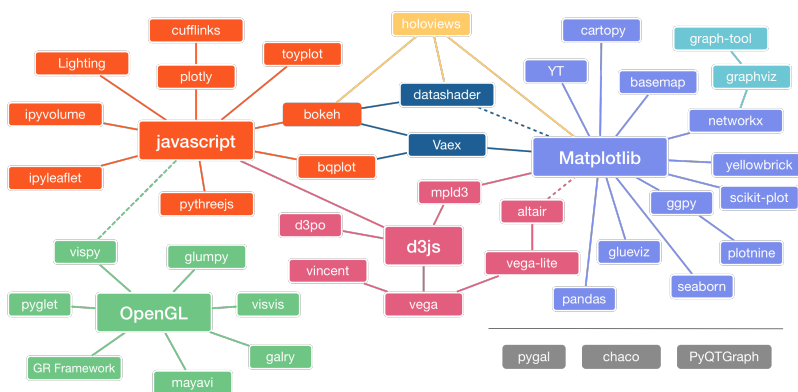


Figure 1: An overview of the *many* plotting packages of Python, and how they relate. Source: <https://rougier.github.io/python-visualization-landscape/>.

0.1 Matplotlib-related packages

Matplotlib has been around for almost two decades and is undoubtedly the most popular plotting library in the Python ecosystem. It is designed primarily with static 2D plots in mind.¹ If all you're looking for is, say, a PNG or PDF figure to go in a report, then matplotlib is an excellent choice whether you are a beginner or an expert with Python, as:

- It boasts a reliable and stable plotting interface.
- The figure customisation options are incredibly flexible.
- Matplotlib has an active development community and comprehensive documentation.²

Many popular packages utilise matplotlib's functionality. For example, the data handling library Pandas includes functions to plot directly from `DataFrame` objects. With so many active matplotlib users, it's also very easy to get help on sites like Stack Overflow or even just by asking a colleague!



¹ By 'static', we mean non-interactive visualisations.

² <https://matplotlib.org/>

Alternatives to matplotlib

For all its strengths, matplotlib *does* have a few downsides. For a start, it can almost be *too* flexible! The default appearance of plots may not be particularly appealing, and generating a publication-worthy figure can be non-trivial and require many lines of code.

Many alternative plotting libraries have been developed which are built on top of matplotlib, improving on its functionality in a number of ways. Among the most popular are:

- *Seaborn*: This fantastic package allows complex data visualisations to be created simply and easily, whilst improving on the default look and feel of matplotlib figures. It also includes a large number of data visualisation functions, including automated regression fitting.
- *Plotnine*: A Python implementation of *The Grammar of Graphics*, which should feel instantly familiar to anyone that has used the popular R package {ggplot2}.



0.2 Javascript-related packages

These packages leverage the power of JavaScript to make interactive 2D plots and animations, and are a great option if you want to create a nifty figure for a website or powerpoint presentation.

One of the most popular examples for data science is plotly. This fantastic library can be used to create visually appealing graphics which are interactive by default, allowing the user to inspect their data via mouse navigation. Plotly is also compatible with many other languages and tools, including R, Julia and MATLAB.



0.3 Other options

- *OpenGL-related packages*: These are a great option if you wish to produce interactive 3D figures. Some examples include PyOpenGL, visvis and pyglet.
- *D3.js-related packages*: These can be used to create interactive data visualisations which are optimised for the web, from declarative JSON specifications. Examples include mpld3 (which brings together D3.js and matplotlib) and Vega-Altair.

0.4 Course overview

This course will be focused on creating static 2D visualisations. We'll start with an in-depth look at the most widely used plotting library matplotlib, covering plotting basics, complex multi-panel plots and customisation. We will then explore the seaborn package, covering a wide range of plot types and visualisation functions which are geared towards data science.

Plotting in Matplotlib

Matplotlib is one of the most widely used and well established visualisation packages for Python. Along with its standing as a pillar of visualisation in the Python world comes comprehensive documentation, an active development community, and a reliable and stable plotting interface. With Matplotlib you can create plots for publications and reports, customise visual style and layouts, make interactive figures, and export your plots to different file formats.

Matplotlib has a number of different interfaces which we may use to construct plots. By far the most popular of these interfaces is the `pyplot` interface. By convention we import this interface with the alias `plt`:

```
import matplotlib.pyplot as plt
```

1.1 Basic plotting

A simple line plot can be created with the function `plt.plot()`. For example, to plot the temperature in Newcastle on a certain (particularly hot) week in August:

```
day = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
temp_newc = [23, 27, 29, 29, 25, 26, 26]

plt.plot(day, temp_newc)
```

Axis labels

Arguably, no plot should exist without axis labels. Otherwise, how would we know what is being shown on the axes? We can correct this by adding axis labels to our plot with `plt.ylabel()` and `plt.xlabel()`. A title can also be added with `plt.title()`:

```
plt.plot(day, temp_newc)

plt.ylabel("Temperature /  $^{\circ}\text{C}$ ")
plt.xlabel("Day")
plt.title("Newcastle Heatwave")
```

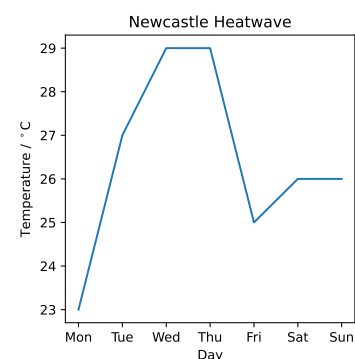
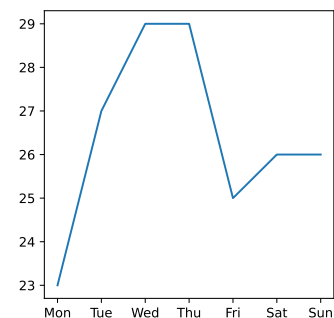


Figure 1.2: A simple line plot with axis labels and a title. `$`'s are used within `plt.ylabel()` to render text in “math mode”. You can read more about writing mathematical expressions in matplotlib at: <https://matplotlib.org/stable/tutorials/text/mathtext.html>.

Axis limits

Along with labelling our axes, we may also set the limits of the x and y axes with the `plt.xlim()`, and `plt.ylim()` functions respectively. In our example we can set the limits of the y -axis:

```
plt.plot(day, temp_newc)

plt.ylim(ymin=20, ymax=30)
plt.ylabel("Temperature /  $^{\circ}\text{C}$ ")
plt.xlabel("Day")
plt.title("Newcastle Heatwave")
```

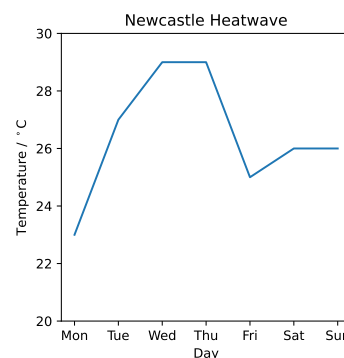


Figure 1.3: A simple line plot with a set y -axis range.

Legends

Multiple lines can be added to one plot by chaining together `plt.plot()` commands. Labels can be added to each line and displayed with `plt.legend()`:

```
temp_edin = [22, 25, 28, 28, 21, 20, 19]

plt.plot(day, temp_newc, label="Newcastle")
plt.plot(day, temp_edin, label="Edinburgh")

plt.ylabel("Temperature /  $^{\circ}\text{C}$ ")
plt.xlabel("Day")
plt.title("UK Heatwave")

plt.legend()
```

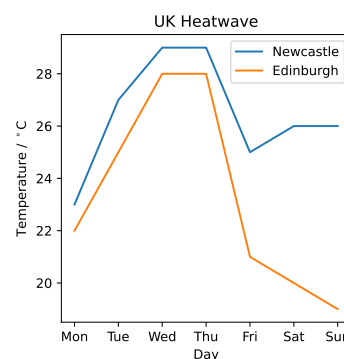


Figure 1.4: A simple line plot with a legend.

By default, `plt.legend()` tries to work out some “best” position to place the legend. However, we can also control the position of the legend with location strings:

```
plt.plot(day, temp_newc, label="Newcastle")
plt.plot(day, temp_edin, label="Edinburgh")

plt.ylabel("Temperature /  $^{\circ}\text{C}$ ")
plt.xlabel("Day")
plt.title("UK Heatwave")

plt.legend(loc="lower left")
```

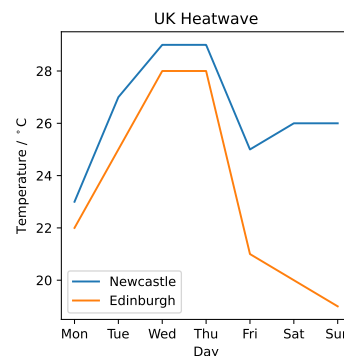


Figure 1.5: A simple line plot with a legend in the lower left corner.

The strings `"upper left"`, `"upper right"`, `"lower left"`, and `"lower right"` place the legend at the corresponding corner of the figure. The strings `"upper center"`, `"lower center"`, `"center left"`, and `"center right"` place the legend at the center of the corresponding edge of the figure. The string `"center"` places the legend at the center of the figure.

Now that we have a `Figure` and `Axes`, we're ready to start plotting. This is done with the `Axes.add_plot()` method. Here we have used the `numpy` package and created a simple sine curve:

```
import numpy as np
x = np.linspace(0, 2*np.pi, 100)
y = np.sin(x)

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(x, y)
```

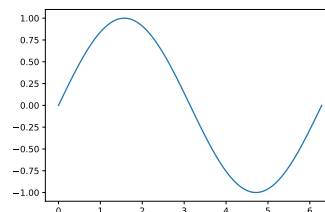


Figure 1.8: A simple plot created with Matplotlib's object-oriented interface.

Axis labels and limits

The `Axes` class has methods `.set_xlabel()` and `.set_ylabel()` with which you can add axis labels to your figure:

```
ax.set_xlabel("x")
ax.set_ylabel("sin(x)")
```

Additionally, we can change the axis limits with the `.set_xlim()` and `.set_ylim()` methods. For example, it would be nice if the x -axis of our plot was between 0 and 2π , and the y -axis between -1 and 1 , the range of the sine function:

```
ax.set_xlim(0, 2*np.pi)
ax.set_ylim(-1, 1)
```

Finally we can give our plot a title with the `.set_title()` method:

```
ax.set_title("A sine curve")
```

Putting this all together gives us the following figure:

```
import numpy as np

x = np.linspace(0, 2*np.pi, 100)
y = np.sin(x)

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(x, y)

ax.set_xlim(0, 2*np.pi)
ax.set_ylim(-1, 1)

ax.set_xlabel("x")
ax.set_ylabel("sin(x)")
ax.set_title("A sine curve")
```

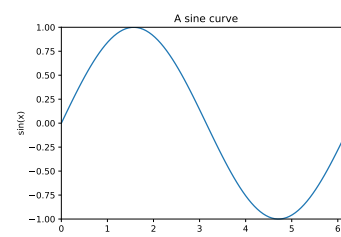


Figure 1.9: A sine curve created with Matplotlib's object-oriented interface; axis labels set with `.set_xlabel()` and `.set_ylabel()`; axis limits set with `.set_xlim()` and `.set_ylim()`, and plot title set with `.set_title()`.

Tick labels

Our figure is starting to come together; it now has axis labels; a title, and is displayed over an appropriate axes range. However, one thing it would be good to improve are the tick labels themselves. For example, we know the key points along the x -axis of this curve in terms of π . We can set specific

tick locations and labels with the `.set_xticks()` and `.set_yticks()` methods.

```
ax.set_xticks(
    ticks=[0, np.pi/2, np.pi, 3*np.pi/2, 2*np.pi],
    labels=[0, "$\pi/2$", "$\pi$", "$3\pi/2$", "$2\pi$"]
)
ax.set_yticks(
    ticks=[-1, 0, 1],
    labels=[-1, 0, "+1"]
)
```

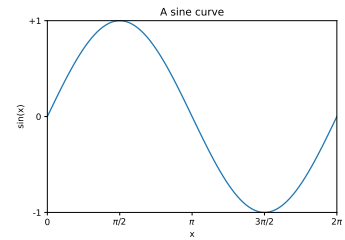


Figure 1.10: A sine curve with well-formatted ticks.

Saving figures

With the object oriented interface we can save our figure with the `.savefig()` method:

```
fig.savefig("sine.pdf")
```

Different types of plots

So far we have only looked at line plots. However, Matplotlib allows us to create many different types of plots. For each type of plot the labels, title, limits etc are all controlled in the same way as above.

Scatter plots

We can create a scatter plot with the `.scatter()` method. We pass our `x` and `y` data to `.scatter()` in the same way as with the `.plot()` method. In addition, we can specify the size, colour and style of the markers with the `s`, `c` and `marker` parameters respectively:

- The `c` parameter can take certain named colours as strings (Table 1.1), or alternatively can be specified with an RGB(A) tuple of float values or a hex RGB(A) string;
- The `s` parameter is a float; and
- The `marker` parameter takes a string which refers to a marker style (Table 1.2).

We can make a scatter plot of some randomly generated data as so:

```
x = np.random.normal(0, 1, 100)
y = x + np.random.normal(0, 0.5, 100)

fig, ax = plt.subplots(figsize=(6, 4))

ax.scatter(x, y, c="green", s=3, marker="^")
ax.set_xlabel("x")
ax.set_ylabel("y")
```

Histograms

Histogram plots are useful for plotting distributions of continuous variables. We create a histogram with the `.hist()` method. For example, to plot a histogram of samples drawn from a normal distribution:

Character	Colour
b	blue
g	green
r	red
c	cyan
m	magenta
y	yellow
k	black

Table 1.1: Subset of named colours available in matplotlib. See https://matplotlib.org/stable/gallery/color/named_colors.html for an exhaustive list.

Character	Description
+	plus marker
o	circle marker
x	cross marker
*	star marker
.	point marker

Table 1.2: Subset of marker styles available in matplotlib. See https://matplotlib.org/stable/api/markers_api.html for an exhaustive list.

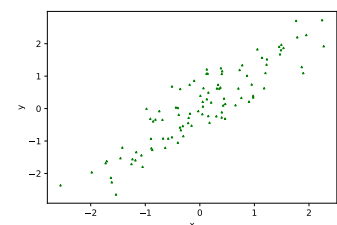


Figure 1.11: A simple scatter plot.

```
s = np.random.normal(size=1000)

fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(s, bins=25, density=True, color="rebeccapurple")

ax.set_xlabel("x")
ax.set_ylabel("density")
```

We can control the number of bins displayed in our histogram plot with the `bins` argument (the default of which is `10`). We can also control whether the y -axis shows frequency or density with the `density` parameter. The colour of our plot can be set with the `color` argument.³

Bar Charts

Another popular plot is the bar chart. For this we use the `.bar()` method. This method also takes a `color` argument. If we want the bars to have different colours then we can set this equal to a list:

```
x = ["Food", "Rent", "Petrol", "Flowers", "Fun"]
spend = [5, 15, 6, 2, 8]
color = [
    "yellowgreen",
    "teal",
    "peru",
    "cornflowerblue",
    "firebrick",
]

fig, ax = plt.subplots(figsize=(6, 4))
ax.bar(x, spend, color=color)

ax.set_xlabel("Category")
ax.set_ylabel("Expenditure (GBP)")
```

Creating subplots

So far we have looked at creating figures that contain only one plot. However, often we will want to display multiple plots together to help convey more information. One way to do this is with `plt.subplots()`. We can assign our figure and axes with:

```
f, axes = plt.subplots(nrows=1, ncols=2, figsize=(8, 4))
```

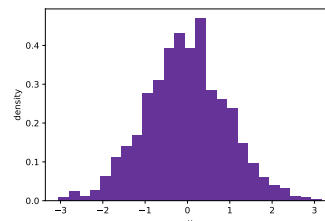


Figure 1.12: A histogram of the normal distribution.

³ Watch out for the American spelling “color” throughout matplotlib!

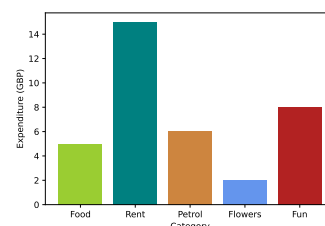
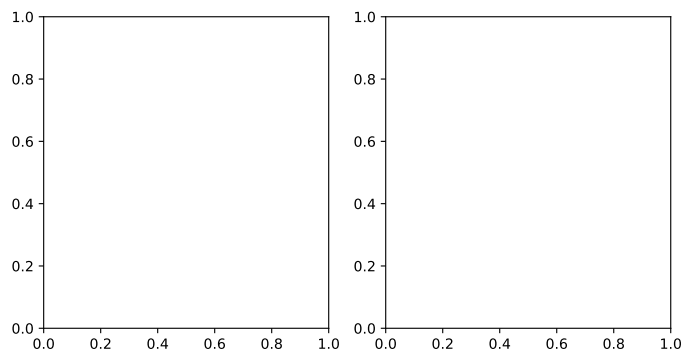


Figure 1.13: A bar chart of monthly expenditure.



This creates an `np.ndarray` object of `matplotlib.axes` objects.

```
type(axes)
#> <class 'numpy.ndarray'>
axes.shape
#> (2,)
type(axes[0])
#> <class 'matplotlib.axes._axes.Axes'>
```

We can work with these `matplotlib.axes` objects as before:

```
x = np.linspace(0, 2*np.pi, 100)
axes[0].plot(x, np.sin(x))
axes[1].plot(x, np.cos(x))
```

Alternatively, we can use tuple unpacking to work with the individual `ax` objects directly:

```
f, (ax0, ax1) = plt.subplots(
    nrows=1, ncols=2, figsize=(8, 4)
)

ax0.plot(x, np.sin(x))
ax1.plot(x, np.cos(x))
```

The `plt.subplots()` method is useful for creating simple subplots. However, if we want to change the width and positions of the subplots it becomes a little more cumbersome.

For example, it is common to plot marginal histogram plots with a scatter plot. To do this requires us to revert to using the `.add_axes()` method. We will first need to define the axes positions for the scatter and marginal plots. We will define the `left`, `bottom`, `width` and `height` of the scatter plot and then use these to get the positions of the marginal plots. We will also define a `spacing` so that the plots don't overlap:

```
left, bottom, width, height = 0.1, 0.1, 0.5, 0.5
spacing = 0.01

rect_scatter = [left, bottom, width, height]
rect_histx = [left, bottom + height + spacing, width, 0.2]
rect_histy = [left + width + spacing, bottom, 0.2, height]
```

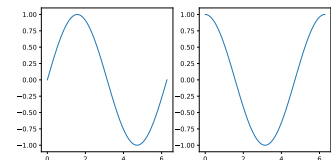


Figure 1.14: Creating a figure with multiple subplots.

We then create our `Figure` and `Axes`. Here we have made use of the `sharex` and `sharey` parameters to ensure that the axes of shared edges match up. Additionally we can turn off the axis labels on our marginal plots with `.tick_params()`⁴

```
fig = plt.figure(figsize=(6, 6))
ax_scatter = fig.add_axes(rect_scatter)
ax_histx = fig.add_axes(rect_histx, sharex=ax_scatter)
ax_histy = fig.add_axes(rect_histy, sharey=ax_scatter)

ax_histx.tick_params(axis="x", labelbottom=False)
ax_histy.tick_params(axis="y", labelleft=False)
```

Then we can add our plots using some randomly generated data as before.

```
x = np.random.normal(0, 1, 100)
y = x + np.random.normal(0, 0.5, 100)

ax_scatter.scatter(x, y)
ax_histx.hist(x)
ax_histy.hist(y, orientation="horizontal")
```

⁴ The `.tick_params()` method has many other parameters which give you control over your axis ticks. We will see more on this in Chapter 2.

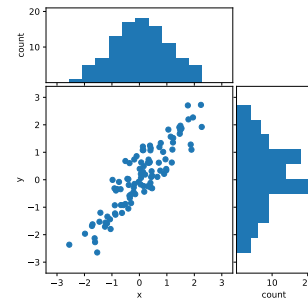


Figure 1.15: A scatter plot with marginal histograms.

1.3 Complex layouts with GridSpec

For more complex subplot layouts it is convenient to use the `GridSpec` class. This can be used to easily specify the geometry of the grid on which to place subplots. If we want to have subplots which span multiple rows or columns, this is much more simple using `GridSpec`.

We start by creating a `Figure` object as before. We then define the grid layout that we want to use with `gridspec.GridSpec()`. This allows us to specify the number of columns and rows of subplots that we want, as well as their width and height ratios.

```
import matplotlib.gridspec as gridspec

fig = plt.figure(figsize=(10, 10))
gs = gridspec.GridSpec(
    nrows=2,
    ncols=2,
    width_ratios=[1, 2],
    height_ratios=[2, 1],
)
```

Now we can add subplots onto this grid using the `add_subplots()` method and specifying their location on the grid with a row and column index, `[row, column]`. To add two subplots in the top right and top left:

```
ax0 = fig.add_subplot(gs[0, 0])
ax0.plot(np.arange(10), color="coral")

ax1 = fig.add_subplot(gs[0, 1])
ax1.plot(np.arange(10), color="seagreen")
```

We are going to have the next subplot span over both columns. We do this by using the slice operator, `:` for the column index.

```
ax2 = fig.add_subplot(gs[1, :])
ax2.plot(np.arange(10), color="orchid")
```

We can actually also use `gridspec` arguments in `plt.subplots()`. This makes it much easier to create subplots of different sizes. In our marginal histogram plot example, instead of defining the rectangles explicitly we could have passed height and width ratios to `plt.subplots()` through the `gridspec_kw` argument. This takes a dictionary of keyword arguments that can be passed to `GridSpec.gridspec()`.

```
fig, ((ax0, ax1), (ax2, ax3)) = plt.subplots(
    2,
    2,
    gridspec_kw={
        "width_ratios": [3, 1],
        "height_ratios": [1, 3],
    },
)
ax1.axis(
    "off"
)
# we do not want to plot anything on this axes
```

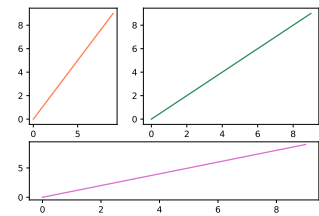
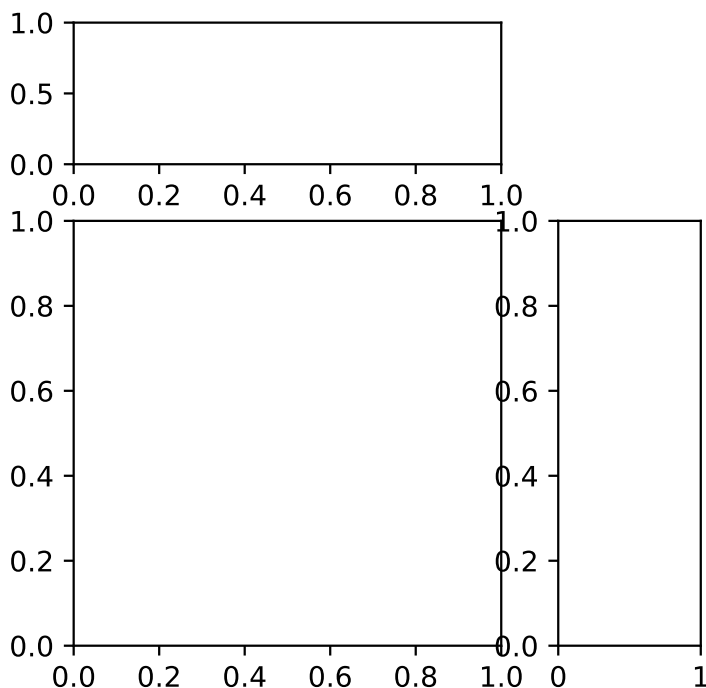


Figure 1.16: A more complex subplot layout.

Figure 1.17: Layout using `plt.subplots()` with `gridspec_kw` arguments.

Customising Plots

As well as being extremely useful for creating exploratory plots, Matplotlib can also be used to build figures for publications and reports. When creating a visualisation to convey information to others, you will want to ensure it is clear, easy to read and visually appealing. You may also wish to ensure that your plots follow certain formatting guidelines set out by your organisation or by a particular journal. In this chapter we will look at how to customise plots in Matplotlib and how we can set out consistent formatting.

2.1 Customising plots as you go

There are many different components of a plot which we can customise. In this section we will cover some of the key plot elements that you may want control over.

Customising fonts

Let's start with a basic plot. For this example we will use some data from the Palmer Penguins dataset which can be imported from Seaborn.¹

```
import matplotlib.pyplot as plt
import seaborn as sns

# Load our dataset
penguins = sns.load_dataset("penguins")

# Select the data we want to plot
penguins = penguins[penguins["species"] == "Gentoo"]
bill_length = penguins["bill_length_mm"]
bill_depth = penguins["bill_depth_mm"]

# Create a scatter plot
fig, ax = plt.subplots(figsize=(5, 5))
ax.scatter(bill_length, bill_depth, c="teal")
```

We can customise the font and font size of any text that our figure contains. We do so with the `fontdict` argument. The methods that we use to control text (i.e those that return `text.Text` instances), such as `.set_title()`, `.set_[xy]label()`, and `.set_[xy]ticklabels()`, all accept this argument. The `fontdict` parameter takes a dictionary of

¹ We will go into more detail on Seaborn in later chapters, but for now we are just using it to access some data to plot.

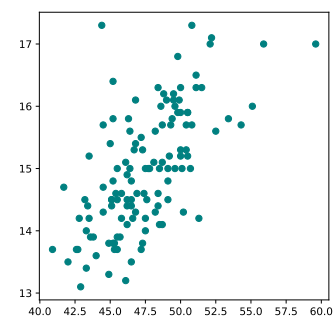


Figure 2.1: A default scatter plot.

keyword arguments for the configuration of a `text.Text` object. We can create a 14pt title in bold italic serif font with:

```
ax.set_title(
    "Bill dimensions of Gentoo Penguins",
    fontdict={
        "fontsize": 14,
        "fontfamily": "serif",
        "fontstyle": "italic",
        "fontweight": "bold",
    },
)
```

Likewise, we can set and format our axis labels with:

```
axis_label_font = {
    "fontsize": 12,
    "fontfamily": "serif",
    "fontstyle": "italic",
}

ax.set_xlabel("Bill Length / mm", fontdict=axis_label_font)
ax.set_ylabel("Bill Depth / mm", fontdict=axis_label_font)
```

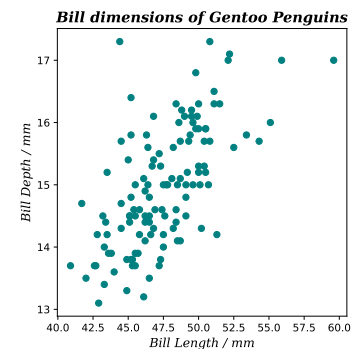


Figure 2.2: A scatter plot with customised labels and title.

Tick parameters

We can control axis tick labels with `.set_[xy]ticklabels()`. These methods also take a `fontdict` argument. Note that we first have to use `.set_[xy]ticks()` to fix the tick positions before formatting their labels:

```
import numpy as np

# Set tick positions
x_tick_locs = np.arange(40, 65, 5)
y_tick_locs = np.arange(13, 18, 1)
ax.set_xticks(x_tick_locs)
ax.set_yticks(y_tick_locs)

axis_tick_font = {"fontsize": 10, "fontfamily": "serif"}
# Format tick labels
ax.set_xticklabels(x_tick_locs, fontdict=axis_tick_font)
ax.set_yticklabels(y_tick_locs, fontdict=axis_tick_font)
```

To skip this step we can actually control label formatting with `.set_[xy]ticks()` directly, as long as we specify the `labels` argument:

```
axis_tick_font = {"fontsize": 10, "fontfamily": "serif"}
ax.set_xticks(
    ticks=x_tick_locs, labels=x_tick_locs, **axis_tick_font
)
```

Chunk 2.1: The arguments of `np.arange()`, follow the `start`, `stop`, `step` pattern we are familiar with from the `range()` function.


```
ax.set_yticks(
    ticks=y_tick_locs, labels=y_tick_locs, **axis_tick_font
)
```

We can further control the appearance of the ticks themselves with `.tick_params()`. In the code below we have added minor ticks to our plot with `.minorticks_on()`, have set the tick direction and size, and have also added ticks to the top and right of the plot.

```
# Turn on minor ticks
ax.minorticks_on()
# Set major tick direction and size,
# and display major ticks on top and right axes
ax.tick_params(
    direction="in",
    which="major",
    length=5,
    top=True,
    right=True,
)
# Set minor tick direction and size,
# and display minor ticks on top and right axes
ax.tick_params(
    direction="in",
    which="minor",
    length=3,
    top=True,
    right=True,
)
```

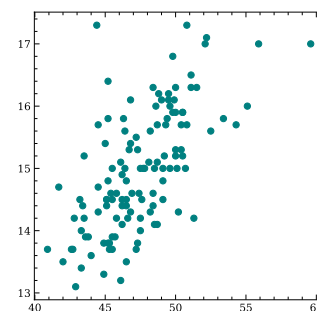


Figure 2.3: A scatter plot with customised ticks.

2.2 Colourmaps

We saw previously how we can change the colour of scatter plot markers with the `c` argument. We can also set the colour of our scatter points to an array like object, for example the body mass of a penguin:

```
body_mass = penguins["body_mass_g"]

fig, ax = plt.subplots(figsize=(5, 5))
ax.scatter(
    bill_length, bill_depth, c=body_mass, cmap="viridis"
)
```

Here we have selected to use the colourmap “viridis” with the `cmap` argument. You can see the list of available colourmaps with:

```
plt.colormaps()
```

Setting the colour of the points on your scatter plot to indicate a variable can be a nice way of adding more information to a figure. However, it is currently not very clear what the colour corresponds to. We need to add a colour bar. This can be done with `fig.colorbar()`. Putting this together

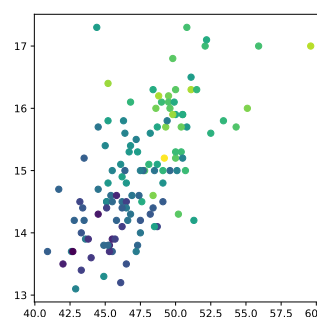


Figure 2.4: A scatter plot where the colour of the markers represent a third variable.

Chunk 2.2: See Figure 2.5 for a visualisation of a subset of these colourmaps.

with our previous formatting:

```

# Create a figure with 2 columns
# We will put our scatter plot onto `ax`
# and our colourbar onto `cax`
fig, (ax, cax) = plt.subplots(
    ncols=2,
    figsize=(8, 7),
    gridspec_kw={"width_ratios": [12, 1]},
)
# Add a scatter plot. We assign this to a
# variable to pass to `fig.colorbar()`.
im = ax.scatter(
    bill_length, bill_depth, c=body_mass, cmap="viridis"
)

# Add a colour bar
cb = fig.colorbar(
    im, # image described by the colorbar
    cax, # axis on which to plot
    orientation="vertical",
    label="Body Mass",
)

# Formatting for scatter plot
ax.set_title(
    "Bill dimensions of Gentoo Penguins",
    fontdict={
        "fontsize": 14,
        "fontfamily": "serif",
        "fontstyle": "italic",
        "fontweight": "bold",
    },
)

# Create a dictionary for reusable label formatting
label_fontdict = {
    "fontsize": 12,
    "fontfamily": "serif",
    "fontstyle": "italic",
}

ax.set_xlabel("Bill Length / mm", fontdict=label_fontdict)
ax.set_ylabel("Bill Depth / mm", fontdict=label_fontdict)

# Formatting for colour bar
cax.set_ylabel(
    "Body Mass",
    rotation=270,
    labelpad=20,

```

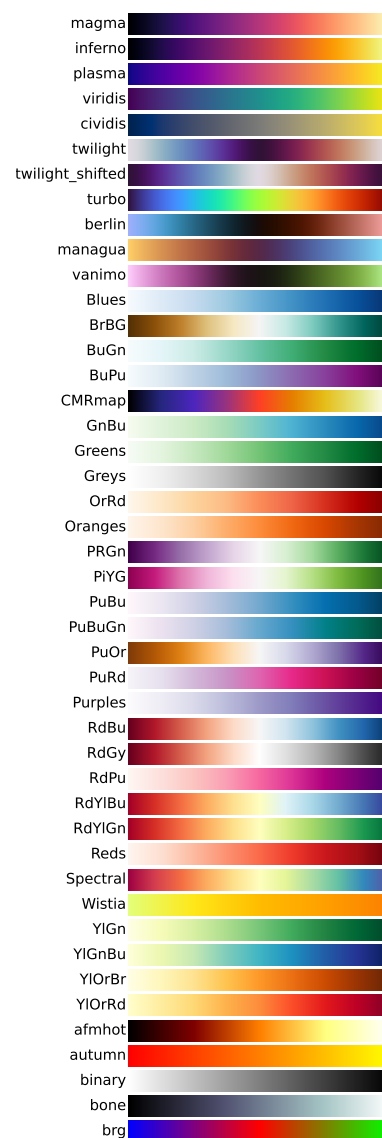
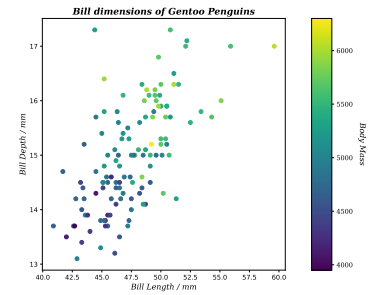


Figure 2.5: A subset of the colourmaps available with Matplotlib. See <https://matplotlib.org/stable/tutorials/colors/colormaps.html> for more.

```

    fontdict=label_fontdict,
)
cax.set_yticks(
    np.arange(4000, 6500, 500),
    np.arange(4000, 6500, 500),
    fontsize=10,
    fontfamily="serif",
)

```



2.3 Runtime configuration parameters

In the previous sections we have looked at how we can customise a particular figure. However, we may have a Python script which creates several figures. If we want these figures to have consistent formatting then we would have to go through and set the formatting options for each figure.

However, it is also possible to change styling settings *globally*. The default Matplotlib styling configuration is set with `matplotlib.rcParams`.² This is a dictionary containing formatting settings and their values. By changing these values we can change default settings, e.g for the font, throughout an entire script or notebook.

To check the parameters available, we can run

```

import matplotlib as mpl
mpl.rcParams

```

If you are making a plot for a publication, a useful thing to do is enable $\text{\LaTeX}$. This is done with the `text.usetex` parameter. We can set this with:

```
mpl.rcParams["text.usetex"] = True
```

Matplotlib will now use $\text{\LaTeX}$ to render text as long as you have $\text{\LaTeX}$ installed. Note that you can use a subset of $\text{\TeX}$ markup in Matplotlib text strings already. Enabling $\text{\LaTeX}$ is more flexible though, and will allow you to use different $\text{\LaTeX}$ packages.³

Another parameter that is useful to set is the font. If you are creating plots for a report then you can set the font of your figures to be consistent with the rest of your document.

```
mpl.rcParams["font.family"] = "serif"
```

To save many lines of formatting settings, multiple parameters that belong to the same group can be set with `matplotlib.rc()`. For the font group, we could set the default family, size and style with:

```
mpl.rc("font", family="serif", size=12, style="italic")
```

If you decide later on to revert back to the standard Matplotlib default settings, you can use:

```
mpl.rcParamsDefaults()
```

² rc is an abbreviation of “runtime configuration”.

³ For more guidance on using $\text{\LaTeX}$ with Matplotlib see the documentation here: <https://matplotlib.org/stable/tutorials/text/usetex.html>.

2.4 Style sheets

Matplotlib comes with a selection of available style sheets. These define a range of formatting parameters and apply them to your figures.

Inbuilt style sheets

You can see which style sheets are available with:

```
plt.style.available
#> ['Solarize_Light2', '_classic_test_patch', '_mpl-
#> gallery', '_mpl-gallery-nogrid', 'bmh', 'classic',
#> 'dark_background', 'fast', 'fivethirtyeight', 'ggplot',
#> 'grayscale', 'petroff10', 'seaborn-v0_8', 'seaborn-v0_8-
#> bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark',
#> 'seaborn-v0_8-dark-palette', 'seaborn-v0_8-darkgrid',
#> 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-
#> notebook', 'seaborn-v0_8-paper', 'seaborn-v0_8-pastel',
#> 'seaborn-v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-
#> ticks', 'seaborn-v0_8-white', 'seaborn-v0_8-whitegrid',
#> 'tableau-colorblind10']
```

This returns a list of keyword strings for accessing a variety of styles. We can set one to use with:

```
plt.style.use("ggplot")
```

Now our plots will have the “ggplot” aesthetic.

```
fig, ax = plt.subplots(figsize=(4, 4))

ax.scatter(bill_length, bill_depth)

ax.set_title("Bill dimensions of Gentoo Penguins")
ax.set_xlabel("Bill Length / mm")
ax.set_ylabel("Bill Depth / mm")
```

Creating your own style sheet

If you have to use the same formatting style repeatedly, for example if you need to apply corporate branding to all your plots, or adhere to a particular journal’s formatting guidelines, it might be convenient to create your own style sheet. This is also useful if you work with a team and want to define a consistent style.

Style sheets are defined in `.mplstyle` files:

```
font.size: 10.0

axes.facecolor: E5E5E5
axes.edgecolor: white
axes.linewidth: 1
axes.grid: True
axes.titlesize: x-large
```

Chunk 2.3: Matplotlib provides a preview of these styles at: https://matplotlib.org/stable/gallery/style_sheets/style_sheets_reference.html.

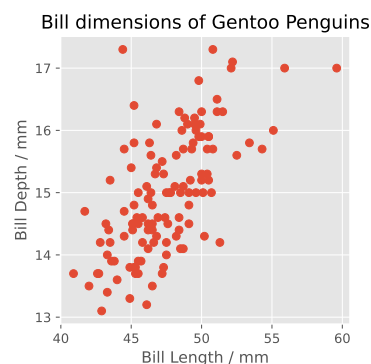


Figure 2.6: A scatter plot with the ggplot theme.

Chunk 2.4: A snippet from the “ggplot” style sheet.

```
axes.labelsize: large
axes.labelcolor: 555555

xtick.color: 555555
xtick.direction: out

ytick.color: 555555
ytick.direction: out
```

We can see that this just contains the values for a variety of runtime configuration parameters.

These files are located within the Matplotlib source code. To find them you can use:

```
import os

os.path.join(mpl.__path__[0], "mpl-data", "stylelib")
```

To load your own style sheet you will first need to create a `<style-sheet-name>.mplstyle` file with your rc settings in the same format as in Chunk 2.4. If you save this file in the same location as the other default styles, you will then be able to use your style sheet in a Python script with:

```
plt.style.use("<style-sheet-name>")
```

If you want to put your style sheet elsewhere, you can still load it into a script by specifying the full or relative path. For example, if I saved my file at `/home/pyviz/fun-style.mplstyle` I would use:

```
plt.style.use("/home/pyviz/fun-style.mplstyle")
```

Plotting with Seaborn

Seaborn is a data visualisation library for Python which builds on the matplotlib package. Having been in development since 2012, it is designed primarily with data exploration in mind. In particular, Seaborn integrates much more closely with pandas data structures and is capable of performing operations on entire datasets. Its visualisation functions are designed to quickly produce detailed and informative statistical plots with few lines of code, making it an invaluable tool for data science with Python.

When importing seaborn, the convention is to use the alias `sns`:¹

```
import seaborn as sns
```

¹ “Why not `sb`?” you might ask; it’s actually an inside joke of the seaborn developers to use the initials of Samuel Norman Seaborn, a fictional character from The West Wing TV series!

3.1 Basic plots

Before we can start making plots, we need some data! As we’ve already seen in the previous chapter, seaborn comes with lots of toy data sets to play around with. We can load the “iris” dataset with:²

```
iris = sns.load_dataset("iris")
iris.head()
#>   sepal_length  sepal_width  petal_length  petal_width
#> species
#> 0          5.1          3.5          1.4          0.2
#> setosa
#> 1          4.9          3.0          1.4          0.2
#> setosa
#> 2          4.7          3.2          1.3          0.2
#> setosa
#> 3          4.6          3.1          1.5          0.2
#> setosa
#> 4          5.0          3.6          1.4          0.2
#> setosa
```

² For a complete list of data sets, see <https://github.com/mwaskom/seaborn-data>.

Running this will generate a pandas `DataFrame` which is assigned to the `iris` variable. Using the `.head()` method, we can inspect the column names and the first five rows of data.

Scatter plots

We will begin with a basic scatter plot of petal length versus sepal length. When working with seaborn’s plotting functions, we always supply our

dataset via the `data` parameter and supply the names of the columns we wish to show:

```
sns.scatterplot(
    data=iris,
    x="sepal_length",
    y="petal_length",
)
```

Notice how seaborn is able to set the axis labels using the column names, so it is not necessary to specify the labels manually!

With seaborn, we are able to control the formatting of the markers—including the colour, shapes and sizes—using the data. We could, for example, distinguish the markers by species using the parameters `hue` (for colour) and `style` (for shape):

```
sns.scatterplot(
    data=iris,
    x="sepal_length",
    y="petal_length",
    hue="species",
    style="species",
)
```

Seaborn also takes care of the legend, giving us a lovely detailed scatter plot with just a single function call!

Before we continue, it's worth noting that we can also set a theme with seaborn. Running `sns.set_theme()`³ will set all subsequent figures to seaborn's "default" theme.⁴ For example:

```
sns.set_theme() # sets all figures to the default theme

sns.scatterplot(
    data=iris,
    x="sepal_length",
    y="petal_length",
    hue="species",
    style="species",
)
```

Our plot now has a grey background with a grid and updated fonts. This can be undone by running:

```
sns.reset_orig()
```

Line plots

To demonstrate line plots, let's first load in another data set:

```
flights = sns.load_dataset("flights")
flights.head()
#>   year month passengers
#> 0  1949   Jan         112
#> 1  1949   Feb         118
```

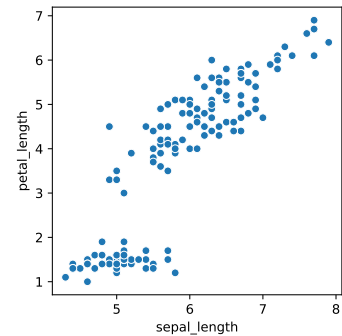


Figure 3.1: Basic scatter plot generated using seaborn.

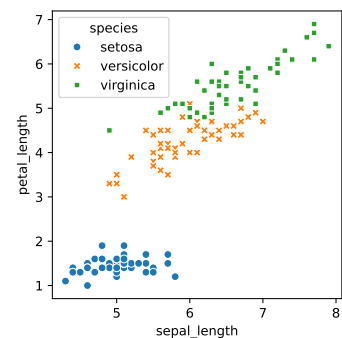


Figure 3.2: Scatter plot with formatted markers generated using seaborn.

³ or `sns.set()` as a shorthand.

⁴ Inspecting the source code of `sns.set_theme()` reveals that it manipulates `mpl.rcParams` to set this theme.

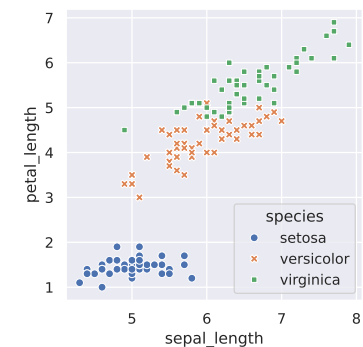


Figure 3.3: Scatter plot generated with seaborn's default theme.


```
#> 2 1949 Mar 132
#> 3 1949 Apr 129
#> 4 1949 May 121
```

This time we will call the `lineplot()` function:

```
sns.lineplot(data=flights, x="year", y="passengers")
```

In addition to a solid line, we also get a shaded error region which, by default, represents the 95% confidence interval.⁵ To remove the error region, we just need to include the argument `ci=None` in the call to `lineplot()`. Alternatively we could set `ci` to an integer to get a specific percentile range, or `"sd"` to display the standard deviation. For example:

```
sns.lineplot(
    data=flights,
    x="year",
    y="passengers",
    ci="sd",
)
```

We can also control the line style, colour, etc using the data. For example, let's use the month column to generate multiple lines with different colours and styles:

```
sns.lineplot(
    data=flights,
    x="year",
    y="passengers",
    hue="month",
    style="month",
)
```

Pretty neat, huh!

3.2 Seaborn vs matplotlib

We have now seen some examples of making basic scatter and line plots with seaborn, but really we've barely scratched the surface! Seaborn comes with a wide variety of functions for more complex data visualisations, which are capable of performing many common statistical operations like regression and kernel density estimation. We will explore these more advanced visualisations in the next chapter.

First though, let's motivate why we believe that seaborn is a good choice for statistical visualisations. To do this, we will create a basic regression plot with seaborn and attempt to replicate it with matplotlib.

Example: scatter plot with a trend line

Let's again make use of the iris dataset loaded earlier. We will make a basic scatter plot showing the petal length versus the sepal length, and overlay with a linear trend line. This is trivial with seaborn:

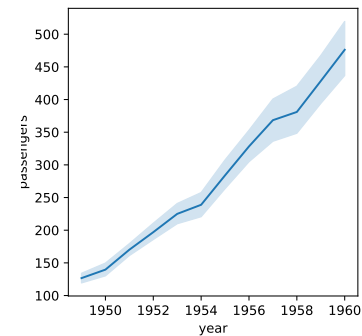


Figure 3.4: Plotting the average number of airline passengers against time, the shaded area representing a 95% confidence interval.

⁵ This is partly why we love seaborn—it does so much of the heavy lifting for us and, unprompted, can visualise lots of statistical features!

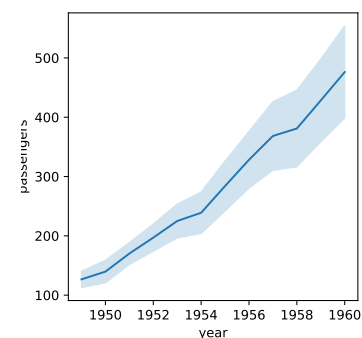


Figure 3.5: Plotting the average number of airline passengers against time, the shaded area representing the standard deviation.

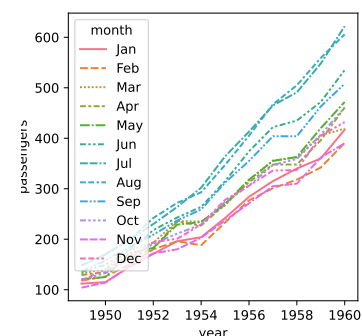


Figure 3.6: Plotting the average number of airline passengers against time, separating by month.

```
sns.regplot(data=iris, x="sepal_length", y="petal_length")
```

With this simple invocation of `sns.regplot()`, we have created a scatter plot with labelled axes and a linear regression line with 95% confidence intervals.

Let's now attempt to make a similar figure with `matplotlib`:

```
import matplotlib.pyplot as plt
import numpy as np

# Calculate the linear relationship
x, y = iris["sepal_length"], iris["petal_length"]
lin = np.polyfit(x, y, 1)
pred = np.poly1d(lin)

# Generate the plot
plt.scatter(x, y)
plt.plot(x, pred(x))
plt.xlabel("sepal_length")
plt.ylabel("petal_length")
```

Aside from the module imports, we've used seven lines of code just to create a basic scatter plot with a trend line, and we haven't even attempted to add a shaded confidence interval! This highlights a number of drawbacks with solely using `matplotlib`:

- `Matplotlib` has no regression functionality, so we have to calculate the linear model separately (in this case we've used `numpy`).
- The data points and the trend line have to be added separately via calls to `plt.scatter()` and `plt.plot()`.
- `Matplotlib` also has no functionality for plotting straight from a `DataFrame`. Instead, we have to feed in the two columns as 1D data structures (in this case `Series` objects).
- Because `matplotlib` cannot access the `DataFrame` labels, we have to supply the axis labels manually using `plt.xlabel()` and `plt.ylabel()`.

Why use `matplotlib`?

At this point, you may be wondering why we would even bother working with `matplotlib` in the first place! It is true that if all you want is a quick statistical plot to visualise and explore your data, `seaborn` is probably the better choice.

However, it is important to note that `seaborn` actually *wraps around* `matplotlib`. In the above example, the `sns.regplot()` function is a wrapper designed to give us a detailed statistical visualisation with much shorter code. This is why the formatting of the two figures here is near-identical: `matplotlib` is used to produce the graphics in both cases!

We have also seen that `matplotlib` is excellent for customisation and fine-tuning of figures, which really matters if you intend to include your figure in a publication or presentation! So it is generally helpful to work with *both*

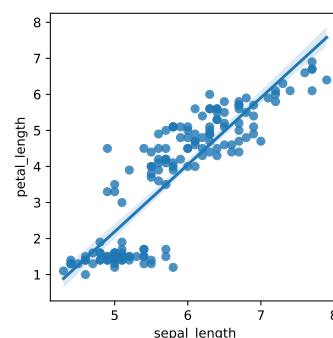


Figure 3.7: Scatter plot with a linear regression line, generated using `seaborn`.

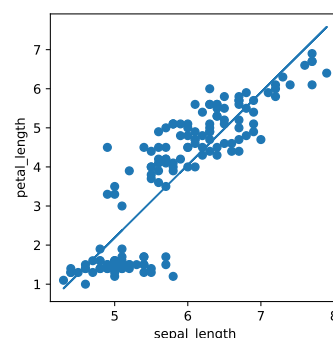


Figure 3.8: Scatter plot with a linear regression line, generated using `matplotlib` and `numpy`.

libraries when creating statistical visualisations.

3.3 Customising seaborn figures with matplotlib

Because seaborn is built on top of matplotlib, we are able to work with both libraries simultaneously. For example, we could initialise a figure using matplotlib, then modify the figure using seaborn. This gives us the best of both worlds: we get matplotlib's flexible customisation together with seaborn's statistical visualisations!

Example: scatter plot with a trend line

Let's have another go at the previous example with the iris data set. First, let's select a more visually appealing style using matplotlib:

```
plt.style.use("seaborn-v0_8")
```

Here we have selected the "seaborn" style sheet, which will set all figures to seaborn's default theme which we saw earlier with `sns.set_theme()`. Let's now reproduce our scatter plot using a combination of matplotlib's object-oriented interface and seaborn's `regplot()` function:

```
fig, ax = plt.subplots(figsize=(5, 8))
sns.regplot(
    data=iris,
    x="sepal_length",
    y="petal_length",
    ax=ax,
)
ax.set_xlabel("Sepal length")
ax.set_ylabel("Petal length")
```

Since we're using the object-oriented interface, we first create `fig` and `ax` objects using `plt.subplots`. To add seaborn's regression plot to our figure, we just need to specify an `ax` argument when calling `sns.regplot()`.

We have then used matplotlib to specify the axis labels; otherwise, the labels would match the `iris` column headings. The fonts and background also match our chosen style sheet. Note that although we have used the seaborn style sheet here, we could have used our own custom sheet (see Chapter 2).

Example: multi-plot figure

We will also consider a multi-plot figure. Let's keep it simple with a two panel figure initialised with `plt.subplots()`:⁶

```
fig, ax = plt.subplots(
    nrows=1, ncols=2, figsize=(5, 8), sharey=True
)
sns.regplot(
    data=iris,
```



Figure 3.9: Scatter plot with a linear regression line, generated using matplotlib and seaborn.

⁶ We will create some more complex multi-plot grids with seaborn in the next chapter!

```

    x="sepal_length",
    y="petal_length",
    ax=ax[0]
)
sns.regplot(
    data=iris,
    x="sepal_width",
    y="petal_length",
    ax=ax[1]
)
ax[0].set_xlabel("Sepal length")
ax[1].set_xlabel("Sepal width")
ax[0].set_ylabel("Petal length")
ax[1].set_ylabel(None)

```

Here, we have used two calls to `sns.regplot()` to add regression plots to each panel. We have then used matplotlib to adjust the axis labels.

Finally, to save the image you can just use `fig.savefig()`.

Standard workflow

The previous examples motivate a sensible workflow when working with seaborn:

1. Set a style using matplotlib
2. Initialise the figure using matplotlib's object-oriented interface
3. Insert the desired statistical visualisations using seaborn
4. Customise and fine-tune the figure using matplotlib

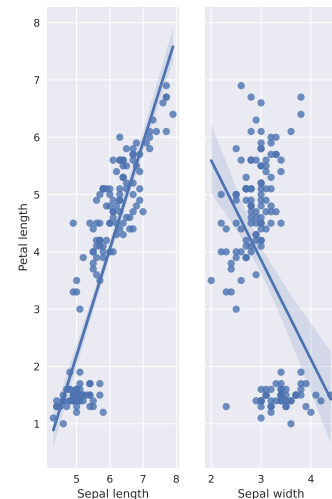


Figure 3.10: Two-panel plot generated using matplotlib and seaborn.

Advanced Statistics with Seaborn

Seaborn has a fantastic array of functions for data visualisation and statistical operations. We will explore a broad selection of these in this chapter, covering bivariate relationships, distributions and categorical figures, before finishing with multi-panel plots including facet and pair grids.

Throughout this chapter, we will adopt the Seaborn theme by running the code below:

```
import matplotlib.pyplot as plt
plt.style.use("seaborn-v0_8")
```

We will continue following the suggested convention at the end of Chapter 3; namely, initialising the `Figure` and `Axes` objects using `matplotlib`, then supplying the `Axes` object using the `ax` argument in the Seaborn plotting functions. This will make it easier to customise the final figure.

4.1 Bivariate relationships

We will start with visualising bivariate relationships, by which we mean two variables plotted against each other. We have already seen examples of scatter and line plots in the previous chapter, but will go into a bit more detail here.

Scatter plots

Let's revisit the penguins data set:

```
import seaborn as sns
penguins = sns.load_dataset("penguins")
```

We will first visualise the bill length versus the flipper length as a scatter plot:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.scatterplot(
    data=penguins,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="species",
    ax=ax,
)
```

Here we have coloured the data points based on the species. We can actually

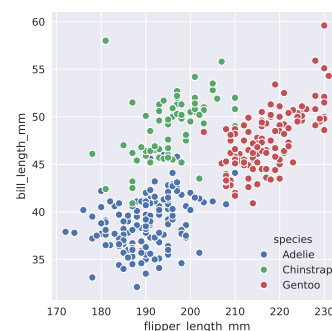


Figure 4.1: Scatter plot with points coloured by species.

map up to three variables using the semantic parameters `hue`, `size` and `style`:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.scatterplot(
    data=penguins,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="species",
    style="island",
    size="body_mass_g",
    ax=ax,
)
```

The point style is now based on island, a categorical variable, whereas the point size is based on body mass which is a numerical variable. For the latter, a linear scale is displayed in the legend for six point sizes.

The colouring in this figure is the default for a categorical variable. If we instead colour by a numerical variable like body mass, a sequential colour gradient will be used:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.scatterplot(
    data=penguins,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="body_mass_g",
    ax=ax,
)
```

Seaborn has a range of colour palettes to choose from.¹ The colour map name can be passed as a string to the `palette` argument:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.scatterplot(
    data=penguins,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="body_mass_g",
    palette="viridis",
    ax=ax,
)
```

In this example we have used the viridis colour map, which varies from dark purple to bright yellow.

Optimisation functions

We have already briefly discussed the `regplot()` function for generating regression fits. Here is an example for body mass versus flipper length:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.regplot(
```

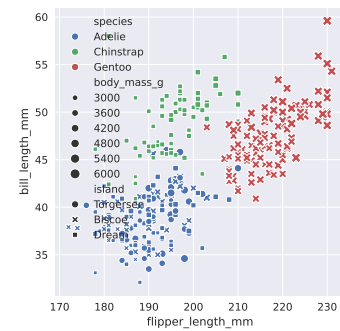


Figure 4.2: Scatter plot with data points of varying size, style and colour.

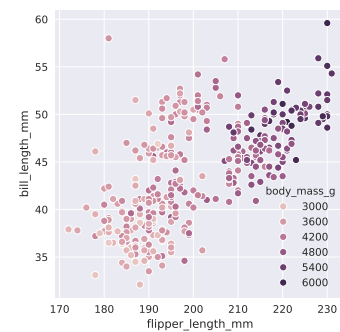


Figure 4.3: Scatter plot coloured by a numerical variable.

¹ For a list of sequential colour palettes, see <https://seaborn.pydata.org/tutorial/color-palettes.html>.

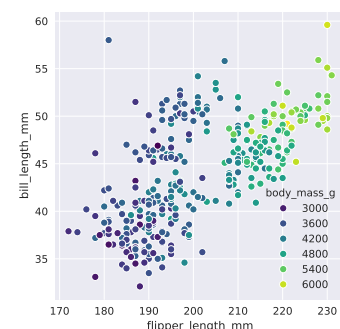


Figure 4.4: Scatter plot coloured by a numerical variable.

```

data=penguins,
x="flipper_length_mm",
y="body_mass_g",
ax=ax,
)

```

This displays a linear fit with the 95% confidence intervals. It's also possible to apply a polynomial fit via the `order` parameter:

```

fig, ax = plt.subplots(figsize=(5, 5))
sns.regplot(
    data=penguins,
    x="flipper_length_mm",
    y="body_mass_g",
    order=2,
    ax=ax,
)

```

In this case, we have an order-2 polynomial (quadratic) fit. One way to assess the quality of our fit is by checking the residuals, which can be obtained using the `residplot()` function:

```

fig, ax = plt.subplots(figsize=(6, 5))
sns.residplot(
    data=penguins,
    x="flipper_length_mm",
    y="body_mass_g",
    order=2,
    ax=ax,
)

```

Seaborn's regression functionality goes even deeper than discussed here, and its online tutorial is a great place to find out more: <https://seaborn.pydata.org/tutorial/regression.html>.

4.2 Distributions

It is often useful to visualise and model the spread of your data. We will now go through some examples of visualising univariate and bivariate distributions.

Histograms

Staying with the penguins data set, let's plot a histogram of the body mass spread:

```

fig, ax = plt.subplots(figsize=(5, 5))
sns.histplot(
    data=penguins,
    x="body_mass_g",
    bins=18,
    ax=ax,
)

```

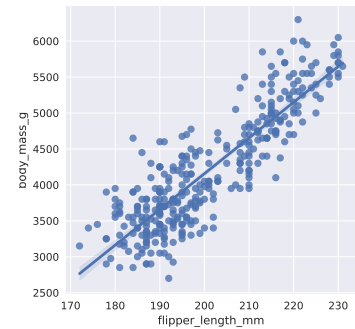


Figure 4.5: Linear fit of body mass vs flipper length.

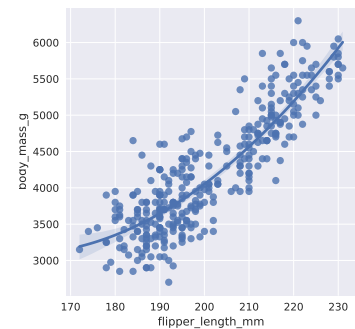


Figure 4.6: Quadratic fit of body mass vs flipper length.

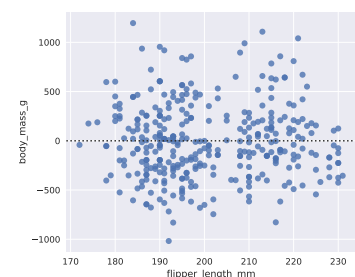


Figure 4.7: Residuals from quadratic fit.

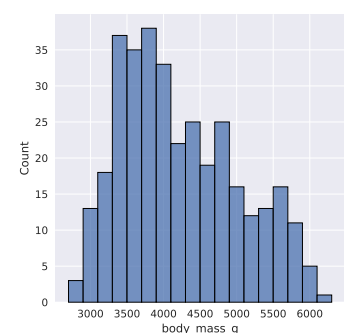


Figure 4.8: Histogram showing the spread in body mass.

The bin size can be varied using either the `binwidth` parameter or by updating the number of bins using the `bins` parameter, as we have done here.

As with the scatter plots above, we can use `hue` to split the histogram by some variable. Let's again use species:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.histplot(
    data=penguins,
    x="body_mass_g",
    bins=18,
    hue="species",
    ax=ax,
)
```

We now have several overlapping histograms and the readability could be improved. One option is to use a stepped histogram by setting `element="step"`, so that we only plot the outline of each histogram:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.histplot(
    data=penguins,
    x="body_mass_g",
    bins=18,
    hue="species",
    element="step",
    ax=ax,
)
```

It's now a bit easier to visually discern the histograms from one another.

Kernel density estimation

Kernel density estimation (KDE) is used to obtain a continuous estimate of a distribution by smoothing histogram counts using a Gaussian kernel. We can generate such a plot using the `kdeplot()` function:

```
fig, ax = plt.subplots(figsize=(6.5, 5))
sns.kdeplot(
    data=penguins,
    x="body_mass_g",
    hue="species",
    bw_adjust=1,
    ax=ax,
)
```

The accuracy of the estimated density is controlled by the bandwidth (the spread) of the Gaussian kernel, which can be adjusted using the `bw_adjust` parameter. A high bandwidth could result in over-smoothing, erasing important features. Meanwhile, a low bandwidth risks introducing random noise in the estimate, like the example below where the bandwidth is set to 0.25:

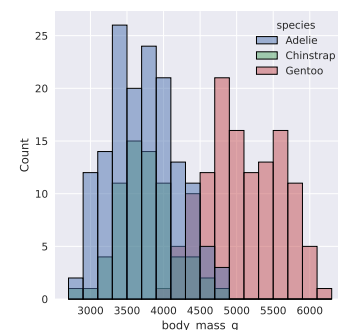


Figure 4.9: Histograms coloured by penguin species.

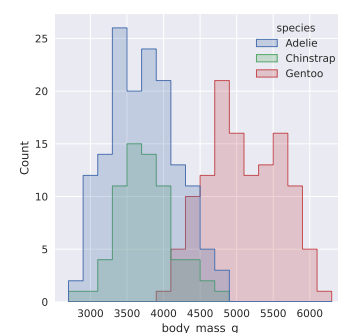


Figure 4.10: Stepped histograms coloured by penguin species.

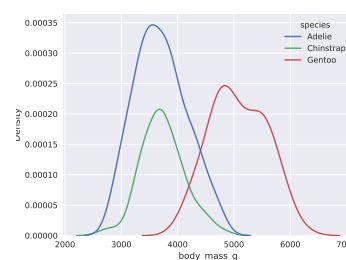


Figure 4.11: KDE of body mass spread coloured by penguin species.


```
fig, ax = plt.subplots(figsize=(6, 5))
sns.kdeplot(
    data=penguins,
    x="body_mass_g",
    hue="species",
    bw_adjust=0.25,
    ax=ax,
)
```

It's also worth noting that the KDE calculation assumes our data is continuous and unbounded. If, for example, it is unrealistic to have values below 0, we can also set `cut=0` when calling `kdeplot()`. This will ensure that the KDE only spans positive values.

Finally, it is possible to plot a histogram together with the KDE by setting `kde=True` when calling `histplot()`:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.histplot(
    data=penguins,
    x="body_mass_g",
    bins=18,
    hue="species",
    element="step",
    kde=True,
    ax=ax,
)
```

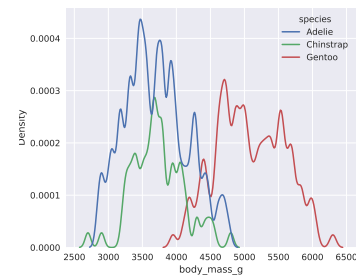


Figure 4.12: KDE plot with a low band-width.



Figure 4.13: Stepped histograms plotted with KDE estimates.

Bivariate distributions

We will now consider bivariate distributions, where we plot the 2D distribution against a pair of variables. This is a great way to visualise the spread of the data and also to check the correlation between variables.

This is really straightforward with Seaborn. We just take the code we've already been using to create histograms and include a `y` variable:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.histplot(
    data=penguins,
    x="body_mass_g",
    y="flipper_length_mm",
    ax=ax,
)
```

We see immediately that the two variables are correlated, since their distribution forms an elongated shape. The colouring indicates the density of observations in each 2D bin, with darker shades implying a higher density. To display a colourbar showing the mapping between counts and colour intensity, we can use `cbar=True`:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.histplot(
```

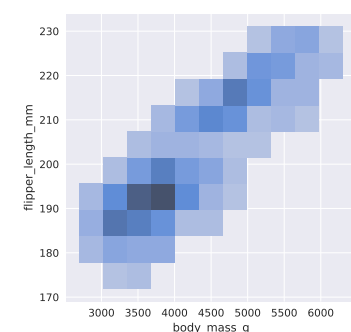


Figure 4.14: Bivariate distribution of body mass vs flipper length.

```

data=penguins,
x="body_mass_g",
y="flipper_length_mm",
cbar=True,
ax=ax,
)

```

We can vary the size of the 2D bins using the `binwidth` argument, which accepts a length-2 tuple with the `x` and `y` widths. For example, `binwidth=(200, 3)` would generate bins with width 200 and height 3.

Let's display the KDE instead, which for a 2D distribution appears as contours:

```

fig, ax = plt.subplots(figsize=(5, 5))
sns.kdeplot(
    data=penguins,
    x="body_mass_g",
    y="flipper_length_mm",
    ax=ax,
)

```

Each contour is drawn at an *iso-proportion* of the density, meaning that it traces a boundary of constant density. The levels are chosen such that the $mass^2$ outside the contour is a particular fraction of the total mass. The levels can be specified explicitly using the `levels` argument. For example:

```

fig, ax = plt.subplots(figsize=(5, 5))
sns.kdeplot(
    data=penguins,
    x="body_mass_g",
    y="flipper_length_mm",
    levels=[0.01, 0.1, 0.5, 0.8],
    ax=ax,
)

```

will draw a KDE plot with levels 1% (outermost contour), 10%, 50%, and 80% (innermost contours).

4.3 Categorical data

We have already seen some approaches for mapping categorical variables, like colouring the data or varying the shape and size. In this section, we will now consider some plots that are specific to categorical data.

Let's start with a scatter visualisation of flipper length versus the categorical species variable. We will use the `swarmplot()` function:

```

fig, ax = plt.subplots(figsize=(5, 5))
sns.swarmplot(
    data=penguins,
    x="species",
    y="flipper_length_mm",
    hue="sex",
)

```

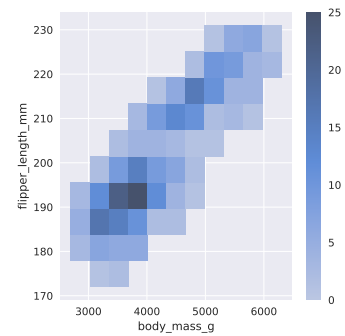


Figure 4.15: Bivariate distribution with colour bar.

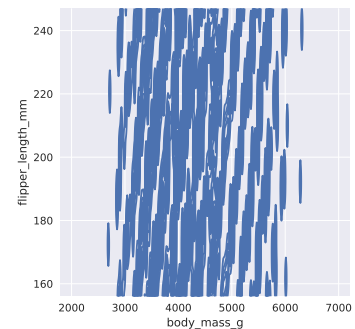


Figure 4.16: Bivariate KDE plot.

² By mass, we mean the integral of the density.

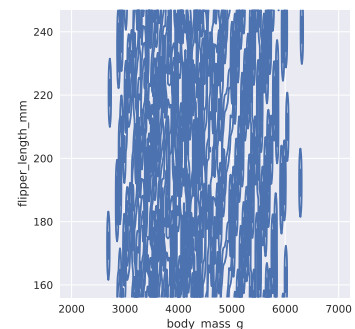


Figure 4.17: Bivariate KDE plot with contours at specified thresholds.

```
size=4,
ax=ax,
)
```

Notice how the data points are shifted horizontally such that there is no overlap. This gives an idea of the distribution of the data in each category. We still have access to the parameters `hue`, `size` and `style`, and here we have coloured the data points by sex.

If we are primarily interested in the spread of the points, we could instead use a box plot:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.boxplot(
    data=penguins,
    x="species",
    y="flipper_length_mm",
    hue="sex",
    ax=ax,
)
```

Box plots display the median (central horizontal line), the interquartile range (spanned by the box) and the minimum/maximum values excluding outliers (the tips of the whiskers). Any outliers are displayed as data points.

And, of course, we can create the classic bar plot! We will demonstrate an example using the flights data set:

```
flights = sns.load_dataset("flights")
flights.head()
#>   year month passengers
#> 0  1949  Jan       112
#> 1  1949  Feb       118
#> 2  1949  Mar       132
#> 3  1949  Apr       129
#> 4  1949  May       121
```

This data set shows the number of passengers each month over 12 years. We can create a bar plot showing the number of passengers each month:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.barplot(
    data=flights,
    x="month",
    y="passengers",
    ax=ax,
)
```

This operates on the entire data set, providing:

- an estimate of the central tendency for each category (in this case the mean)
- an error bar displaying the uncertainty.

Bar plots are a good choice if you wish to compare to the value 0, which is included in the axis range for the quantitative variable.

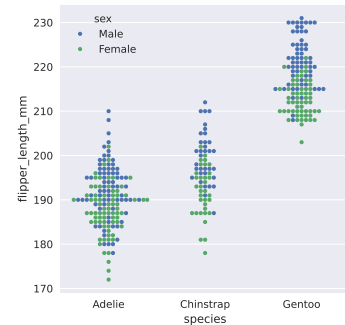


Figure 4.18: Scatter plot of species vs flipper length, coloured by sex.

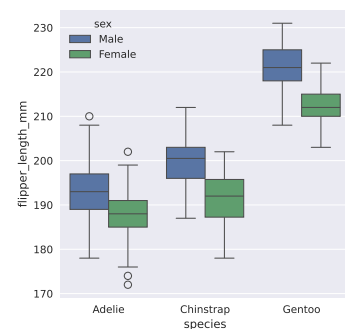


Figure 4.19: Box plot of species vs flipper length, coloured by sex.

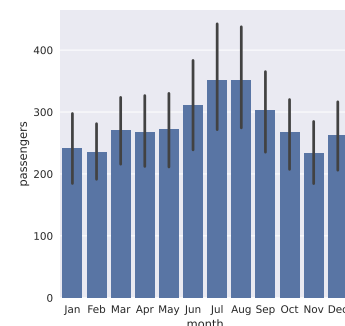


Figure 4.20: Bar plot of month vs passengers.

4.4 Multi-panel plots

We've covered a lot of plots in this chapter! It may feel a bit overwhelming at first, however the plotting functions all accept similar inputs (`x`, `y`, `data`, `hue`, etc), and it should start feeling fairly intuitive once you've made a few visualisations with Seaborn. To finish, we will look at how to construct complex multi-panel figures with Seaborn.

Facet grids

Facet grids can be used to construct multiple plots using subsets of the dataset, which is split based on the values of a couple of variables. Let's take our distribution of penguin body mass from earlier, and create multiple panels based on sex and species using the `FacetGrid()` function:

```
g = (
    sns.FacetGrid(
        data=penguins,
        row="sex",
        col="species",
    )
    .map(
        sns.histplot,
        "body_mass_g",
        element="step",
    )
)
```



Figure 4.21: Facet grid split by penguin species and sex.

The `FacetGrid()` call creates a `FacetGrid` object, which is assigned to the variable `g`. Here we supply the data set we wish to use, along with the variables to use for splitting. In this case, we create two rows based on sex (with the male population shown in the top row and female population shown in the bottom row) and three columns based on species.

We also have to map some plot onto these panels (otherwise they will be empty). We do this using the `.map()` method. The first input will be the

name of the plotting function to use (in this case `sns.histplot()`), then we supply the inputs we would normally use when calling this function on its own.

The result is a facet grid showing the stepped histogram for each subset of the data. It's worth noting that we don't have to use multiple columns and rows here. For example, we could have used a single column and instead coloured by species using the `hue` parameter:

```
g = (
    sns.FacetGrid(
        data=penguins,
        row="sex",
        hue="species",
    )
    .map(
        sns.histplot,
        "body_mass_g",
        element="step",
    )
    .add_legend()
)
```

In this case, we have used `.add_legend()` to include a species legend.

Pair grids

Pair plots are used to show the relationships between every combination of two variables in a data set. In Seaborn, they can be created by defining a `PairGrid` object, then selecting the plot types to show in each panel:

```
g = (
    sns.PairGrid(data=penguins, diag_sharey=False)
    .map_upper(sns.scatterplot)
    .map_lower(sns.kdeplot)
    .map_diag(sns.kdeplot)
)
```

The data set is supplied to the `data` argument in `PairGrid()`. By default, the numerical variables from the data set will be displayed. The output of `PairGrid` is assigned to the variable `g`. This line alone will generate a grid of empty panels. The panels are filled by chaining together some methods of `PairGrid`:

- `.map_diag()` : fills all diagonal panels in the pair grid
- `.map_upper()` and `.map_lower()` : fills the panels above and below the diagonal
- A few other methods not shown here: `.map_offdiag()` fills all off-diagonal panels; `.map()` fills *all* panels.

We just need to provide the plotting function to use in each case. Here, we have selected KDE plots for the diagonal and lower panels and scatter plots for the upper panels. Notice that a univariate KDE is displayed for the diagonal panels. Setting `diag_sharey=False` will ensure that these

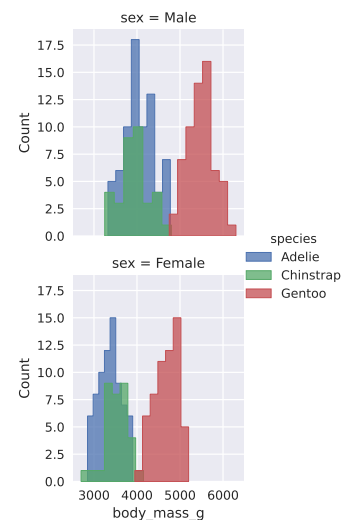


Figure 4.22: Facet grid split by penguin sex and coloured by species.

Chunk 4.1: Code used to create Figure 4.23.

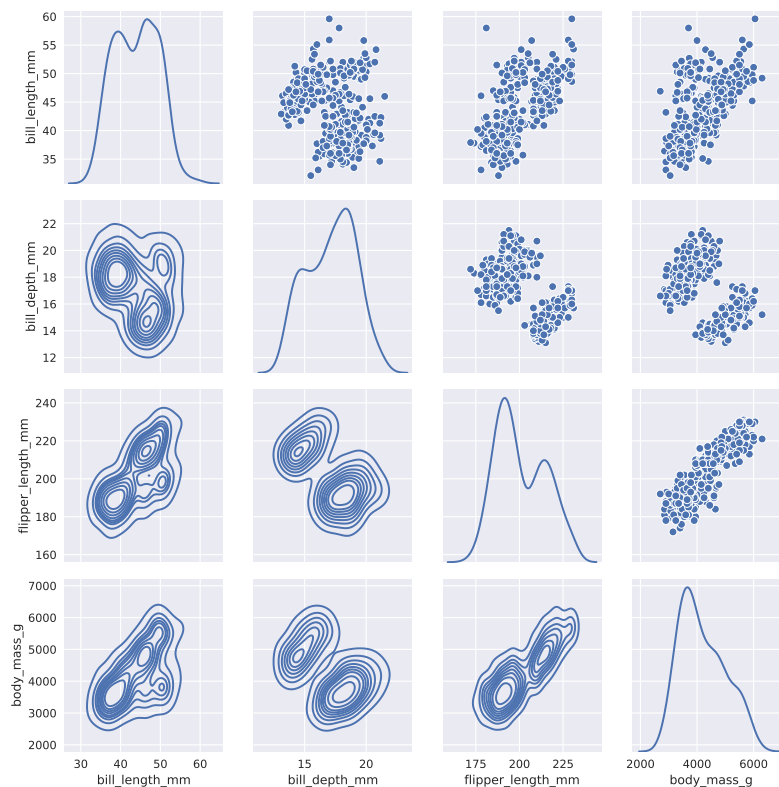


Figure 4.23: Pair grid for the penguins data set (created by Chunk 4.1).

KDEs extend the full height of the vertical axis, since the axes will not be shared with the other panels.

We can also colour the data by one of our categorical variables. To colour by species, we just set `hue="species"` in the call to `PairGrid()` :

```
g = (
    sns.PairGrid(
        data=penguins,
        diag_sharey=False,
        hue="species",
    )
    .map_upper(sns.scatterplot)
    .map_lower(sns.kdeplot)
    .map_diag(sns.kdeplot)
    .add_legend()
)
```

Chunk 4.2: Code used to create Figure 4.24.

The legend is again added by calling `.add_legend()` .

Finally, you may only wish to show a single type of plot in the off-diagonal elements. In this case, you could consider using a corner plot, which is created by setting `corner=True` and filling just the lower panels:

```
g = (
    sns.PairGrid(
        data=penguins,
        diag_sharey=False,
```

Chunk 4.3: Code used to create Figure 4.25.

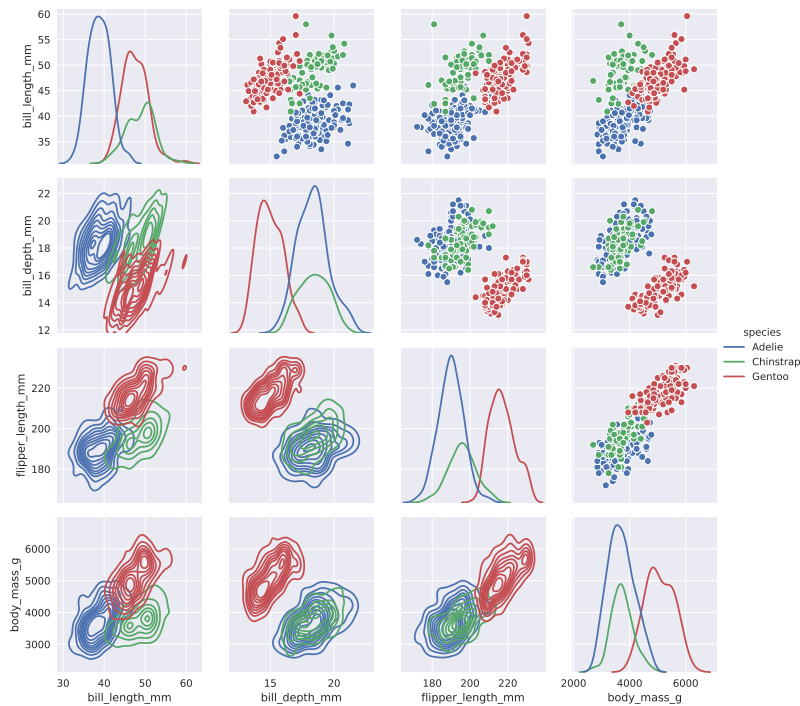


Figure 4.24: Pair grid coloured by penguin species (created by Chunk 4.2).

```

    hue="species",
    corner=True,
)
.map_lower(sns.kdeplot)
.map_diag(sns.kdeplot)
.add_legend()
)

```

Customising multi-panel figures

Customisation of `FacetGrid` and `PairGrid` figures is a little more complicated. There is no option to supply a `matplotlib Axes` object when creating these plots (unlike the other plotting functions discussed so far). This is because `FacetGrid()` and `PairGrid()` both initialise a `matplotlib` figure for us. We therefore need to access the `Figure` and `Axes` objects before we can customise the plots using more familiar `matplotlib` code.

For a `FacetGrid` or `PairGrid` object assigned to variable `g`, the `Figure` and `Axes` objects can be accessed using the `.figure` and `.axes` attributes. For example:

```

g = sns.FacetGrid(
    data=penguins,
    row="sex",
    col="species",
).map(
    sns.histplot,
    "body_mass_g",
    element="step",

```

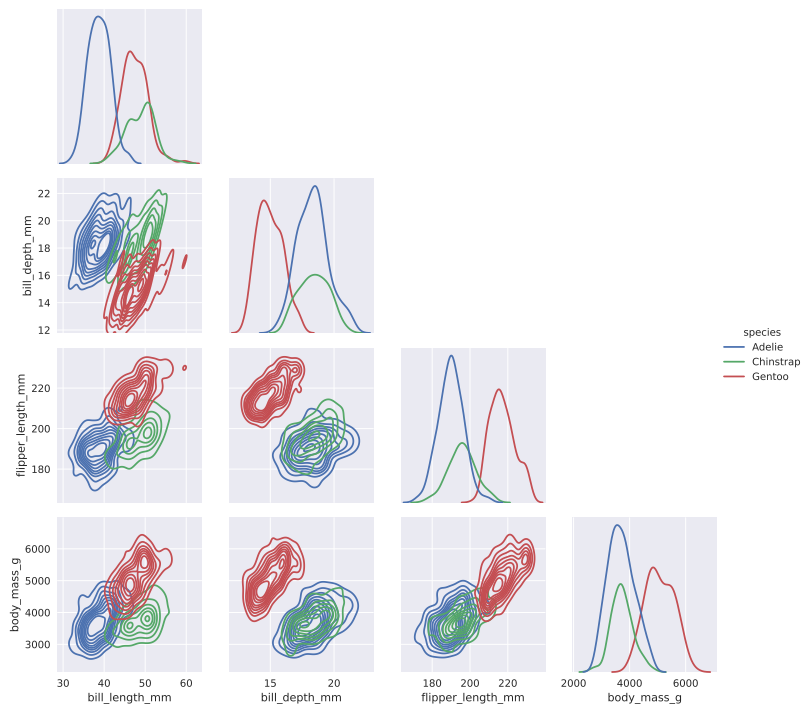


Figure 4.25: Corner plot with univariate and bivariate KDEs (created by Chunk 4.3).

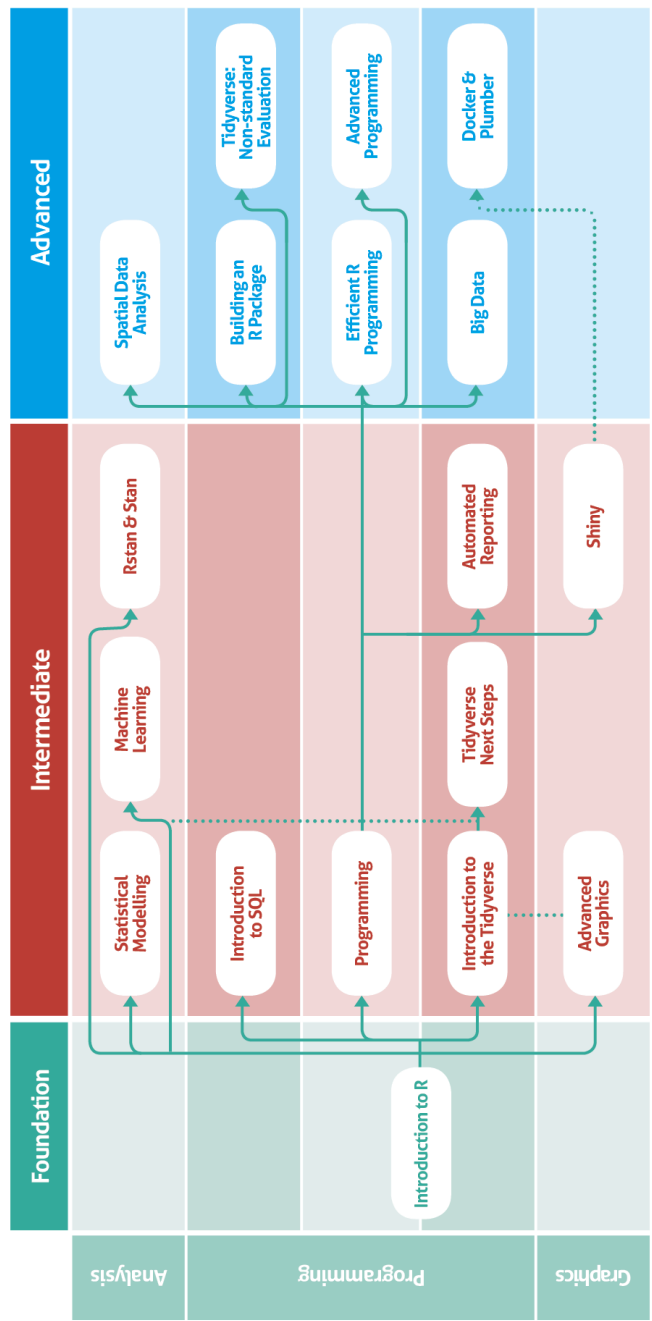
```
)

fig, ax = g.figure, g.axes
for i in range(3):
    ax[1, i].set_xlabel("Body mass [g]")
fig.savefig("grid.pdf")
```

Here, we have created the 2-by-3 facet grid from earlier, then edited the axis labels and saved the figure using matplotlib code.

Before finishing, we *strongly* recommend Seaborn's fantastic plotting tutorials: <https://seaborn.pydata.org/tutorial.html>. These cover all the functions discussed in this chapter in more depth.

Training Course Dependencies





OUR PARTNERS